

# MCT-454L Mobile Robotics

## Lab 6: Reactive Navigation Using TurtleBot3 LiDAR in Gazebo Environment

### 1. Objective

The objective of this lab session was to implement reactive navigation using LiDAR sensor data within the ROS 2 and Gazebo simulation environments. The primary goal was to process raw `/scan` data to perceive the environment and develop autonomous behaviors, including obstacle detection, avoidance, and wall following, without the use of a pre-generated map.

### 2. Methodology & Implementation

A custom ROS 2 Python node (`lidar_navigator`) was developed to process the LaserScan data and publish motion commands.

- **Scan Processing & Region Extraction:** The 360-degree LiDAR data was partitioned into three distinct regions: Front ( $-30^\circ$  to  $+30^\circ$ ), Left ( $30^\circ$  to  $90^\circ$ ), and Right ( $270^\circ$  to  $330^\circ$ ). Invalid measurements (`inf` or `NaN`), which occur when no obstacle is within range, were replaced with the sensor's maximum range to prevent computational errors.
- **Decision-Making Logic:** The minimum distance within each region was computed. If the `front_dist` dropped below the safety threshold (0.5m), the robot was programmed to halt forward velocity instantly. It then compared the `left_dist` and `right_dist`, commanding an angular velocity (0.5 rad/s) toward the region with the greater clearance. If the front was clear, it maintained a steady linear velocity of 0.15 m/s.

### 3. Observations

During the testing of the reactive navigation node, several key behaviors were observed:

- **Robot Behavior Near Obstacles:** The robot successfully halted linear progression when approaching a wall head-on. Because the logic compared the left and right clearances, the robot smoothly chose the optimal escape route rather than turning blindly.
- **Oscillations and Instability:** Slight oscillations were noted when navigating tight corners or narrow corridors. If the robot turned to avoid a front obstacle, it would occasionally detect a new obstacle on its side immediately after, causing it to rapidly switch turning directions. Using the minimum distance (`np.min`) over a wide degree range helped stabilize this compared to relying on a single LiDAR ray.

- **Effect of Changing Threshold Values:**

- *Increasing the threshold* (e.g., to 1.0m) made the robot overly cautious. It successfully avoided collisions but struggled to navigate through narrower gaps or corridors, often turning prematurely.
- *Decreasing the threshold* (e.g., to 0.2m) allowed the robot to navigate tighter spaces, but increased the risk of collision. Due to the robot's physical momentum and the slight delay in publishing velocity commands, stopping too late resulted in grazing the walls.

## **4. Conclusion**

This lab successfully demonstrated how raw sensor data can be translated into meaningful, real-time control decisions. By implementing a reactive navigation algorithm, I learned how to process arrays of range data, handle noisy or infinite sensor values, and apply conditional logic to drive a mobile robot autonomously. The primary challenge was tuning the threshold parameters to find the correct balance between safety and maneuverability, reinforcing the importance of robust control logic in autonomous systems.